SEGUNDO DEPARTAMENTAL- EJERCICIO 7 bis- LISTA DINÁMICA

En esta práctica desarrollamos un programa que permite implementar una lista dinámica, el objetivo es administrar una colección de números enteros mediante una estructura de datos formada por nodos enlazados, los cuales se conectan entre sí mediante apuntadores, a diferencia de una estructura estática, nuestra lista dinámica no necesita establecer un tamaño máximo desde el inicio, ya que podemos agregar elementos conforme los necesitemos, dependiendo de la memoria disponible, además, implementamos diferentes operaciones que nos permiten insertar elementos al inicio y al final, eliminar elementos por su valor, buscar números, mostrar el contenido de la lista y verificar si se encuentra vacía, finalmente, incorporamos un menú interactivo para facilitar el manejo de las operaciones y un destructor que libera la memoria utilizada por los nodos.

Errores y Correcciones: En la versión inicial, al momento de ejecutar en repetidas ocasiones alguna opción del menú la pantalla se llenaba información lo que causaba confusión al momento de comprobar la operación solicitada. Por lo que agregamos una nueva opción al menú mediante la estructura switch: “Limpiar pantalla”.

Código Completo:

```
#include <iostream>
#include <stdlib.h>
using namespace std;

// Clase que representa cada nodo de la lista
class Nodo {
public:
    int dato;
    Nodo* siguiente;
    // Constructor del nodo
    Nodo(int valor) {
        dato = valor;
        siguiente = NULL;
    }
};
// Clase que maneja la lista dinamica
class Lista {
private:
    Nodo* inicio;
```

```
public:
    // Constructor de la lista
    Lista() {
        inicio = NULL;
    }
    // Destructor: libera la memoria de los nodos
    ~Lista() {
        while (inicio != NULL) {
            Nodo* auxiliar = inicio;
            inicio = inicio->siguiente;
            delete auxiliar;
        }
 }
    // Insertar un elemento al inicio
    void insertarInicio(int valor) {
        Nodo* nuevo = new Nodo(valor);

        nuevo->siguiente = inicio;
        inicio = nuevo;

        cout << "Elemento insertado al inicio.\n";
    }
    // Insertar un elemento al final
    void insertarFinal(int valor) {
        Nodo* nuevo = new Nodo(valor);

        if (inicio == NULL) {
            inicio = nuevo;
        } else {
            Nodo* actual = inicio;

            while (actual->siguiente != NULL) {
                actual = actual->siguiente;
            }
```

```
        actual->siguiente = nuevo;
    }
    cout << "Elemento insertado al final.\n";
}
// Eliminar un elemento por su valor
void eliminar(int valor) {
    if (inicio == NULL) {
        cout << "La lista esta vacia.\n";
        return;
    }
    // Si el elemento esta al inicio
    if (inicio->dato == valor) {
        Nodo* auxiliar = inicio;
        inicio = inicio->siguiente;
        delete auxiliar;

        cout << "Elemento eliminado.\n";
        return;
    }
    // Buscar el elemento
    Nodo* actual = inicio;

    while (actual->siguiente != NULL &&
           actual->siguiente->dato != valor) {
        actual = actual->siguiente;
    }
    if (actual->siguiente == NULL) {
        cout << "Elemento no encontrado.\n";
    } else {
        Nodo* auxiliar = actual->siguiente;
        actual->siguiente = auxiliar->siguiente;
        delete auxiliar;
        cout << "Elemento eliminado.\n";
    }
```

```
    }
    // Buscar un elemento
    void buscar(int valor) {
        Nodo* actual = inicio;
        int posicion = 1;
        while (actual != NULL) {
            if (actual->dato == valor) {
                cout << "Elemento encontrado en la posicion "
                     << posicion << ".\n";
                return;
            }
            actual = actual->siguiente;
            posicion++;
        }
        cout << "Elemento no encontrado.\n";
    }
    // Mostrar todos los elementos
    void mostrar() {
        if (inicio == NULL) {
            cout << "La lista esta vacia.\n";
            return;
        }
        Nodo* actual = inicio;
        cout << "\nElementos de la lista:\n";
        while (actual != NULL) {
            cout << actual->dato << " -> ";
            actual = actual->siguiente;
        }
        cout << "NULL\n";
    }
    // Consultar si la lista esta vacia
    bool estaVacia() {
        return inicio == NULL;
    }
};
```

```
int main() {
    Lista lista;
    int opcion, valor;

    do {
        cout << "\n=== LISTA DINAMICA EN POO ===\n";
        cout << "1. Insertar al inicio\n";
        cout << "2. Insertar al final\n";
        cout << "3. Eliminar elemento\n";
        cout << "4. Buscar elemento\n";
        cout << "5. Mostrar lista\n";
        cout << "6. Verificar si esta vacia\n";
        cout << "7. Limpiar pantalla\n";
        cout << "8. Salir\n";
        cout << "Selecciona una opcion: ";
        cin >> opcion;

        switch (opcion) {
            case 1:
                cout << "Ingresa un numero: ";
                cin >> valor;
                lista.insertarInicio(valor);
                break;

            case 2:
                cout << "Ingresa un numero: ";
                cin >> valor;
                lista.insertarFinal(valor);
                break;

            case 3:
                cout << "Ingresa el numero que deseas eliminar: ";
                cin >> valor;
                lista.eliminar(valor);
```

```
                break;

            case 4:
                cout << "Ingresa el numero que deseas buscar: ";
                cin >> valor;
                lista.buscar(valor);
                break;

            case 5:
                lista.mostrar();
                break;

            case 6:
                if (lista.estaVacia())
                    cout << "La lista esta vacia.\n";
                else
                    cout << "La lista contiene elementos.\n";
                break;

            case 7:
                system("cls");
                break;

            case 8:
                cout << "Programa finalizado.\n";
                break;

            default:
                cout << "Opcion no valida.\n";
        }
    } while (opcion != 8);
    return 0;
}
```

Explicación del Código:

1. Declaración de la clase Nodo (class Nodo)

Creamos la clase Nodo, cuya función es representar cada uno de los elementos que forman parte de nuestra lista dinámica, cada nodo almacena un número entero y un apuntador que permite conectarlo con el siguiente nodo de la estructura.

2. Declaración de los atributos de Nodo (dato y siguiente)

- int dato: almacena el número entero que queremos guardar en la lista.
- Nodo* siguiente: es un apuntador que almacena la dirección de memoria del siguiente nodo.

Gracias a estos atributos podemos conectar diferentes nodos y construir nuestra lista dinámica.

3. Constructor de la clase Nodo (Nodo(int valor))

Creamos un constructor que recibe un número entero mediante el parámetro valor, de esta manera, cada vez que creamos un nuevo nodo, sus atributos quedan inicializados correctamente.

4. Declaración de la clase Lista (class Lista)

Creamos la clase Lista, que se encarga de administrar todos los nodos que forman parte de nuestra estructura dinámica, dentro de esta clase definimos los métodos necesarios para insertar, eliminar, buscar y mostrar elementos.

5. Declaración del apuntador principal (Nodo* inicio)

Este apuntador nos permite acceder al resto de los elementos, ya que cada nodo contiene la dirección del siguiente.

6. Constructor de la clase Lista (Lista())

Creamos el constructor de nuestra lista, cuya función es inicializar el apuntador inicio, al asignarle NULL, indicamos que nuestra lista comienza vacía, porque todavía no contiene ningún nodo.

7. Método para insertar al inicio (insertarInicio(int valor))

Creamos el método insertarInicio(), cuya función es agregar un nuevo elemento al principio de nuestra lista. Recibimos un número entero mediante el parámetro valor y creamos un nuevo nodo utilizando:

Nodo* nuevo = new Nodo(valor);

Con new reservamos memoria dinámica para el nodo y ejecutamos su constructor para almacenar el valor recibido.

8. Conexión del nuevo nodo (nuevo->siguiente = inicio)

Con ella hacemos que el nuevo nodo apunte al que anteriormente era el primer elemento de nuestra lista, esto permite conservar la conexión con todos los nodos que ya estaban almacenados.

9. Actualización del inicio (inicio = nuevo)

Con esta instrucción actualizamos el apuntador principal para que ahora señale al nuevo nodo, de esta manera, el elemento recién insertado se convierte en el primero de nuestra lista.

10. Método para insertar al final (insertarFinal(int valor))

Su función es agregar un nuevo elemento al último lugar de nuestra lista, al igual que en la inserción al inicio, recibimos un número entero y creamos un nuevo nodo utilizando new.

11. Verificación de la lista vacía (if (inicio == NULL))

Después de crear el nuevo nodo, verificamos si inicio es igual a NULL, si se cumple esta condición, significa que nuestra lista está vacía, de esta manera, el nuevo nodo se convierte en el primer elemento de la lista.

12. Recorrido de la lista (Nodo* actual = inicio)

Si nuestra lista ya contiene elementos, declaramos un apuntador auxiliar llamado actual y lo inicializamos con la dirección del primer nodo, utilizamos este apuntador para recorrer la lista sin modificar el apuntador principal inicio.

13. Ciclo para encontrar el último nodo (while (actual->siguiente != NULL))

Utilizamos un ciclo while para avanzar por los nodos hasta encontrar el último elemento, la condición actual->siguiente != NULL indica que todavía existe otro nodo después del actual, cuando llegamos al último nodo, su apuntador siguiente contiene NULL y el ciclo termina.

14. Conexión del nuevo nodo al final (actual->siguiente = nuevo)

Con esta instrucción conectamos el último nodo existente con el nuevo nodo, como el constructor inicializó nuevo->siguiente en NULL, el nuevo elemento se convierte en el último de nuestra lista.

15. Método para eliminar elementos (eliminar(int valor))

S función es retirar de nuestra lista un elemento que coincida con el número ingresado por el usuario, recibimos el número mediante el parámetro valor y verificamos primero si nuestra lista contiene elementos.

16. Eliminación del primer elemento (if (inicio->dato == valor))

Antes de buscar el elemento en el resto de la lista, verificamos si el número que queremos eliminar se encuentra en el primer nodo, si inicio->dato == valor, significa que encontramos el elemento al inicio.

17. Preparación para buscar el elemento (Nodo* actual = inicio)

Si el elemento no se encuentra al inicio, declaramos el apuntador actual y lo inicializamos con inicio, este apuntador nos permite recorrer la lista para localizar el nodo que contiene el valor que deseamos eliminar.

18. Búsqueda del nodo anterior (while)

Este ciclo avanza por la lista mientras se cumplan dos condiciones:

- Que exista un siguiente nodo.
- Que el dato del siguiente nodo sea diferente del valor que buscamos.

El objetivo es localizar el nodo que se encuentra antes del elemento que queremos eliminar.

19. Verificación de elemento no encontrado (if (actual->siguiente == NULL))

Cuando termina el ciclo, verificamos si actual->siguiente es igual a NULL, si se cumple esta condición, significa que recorrimos toda la lista y no encontramos el valor solicitado.

20. Eliminación de un elemento intermedio o final (else)

Con esto guardamos la dirección del nodo que vamos a eliminar, después conectamos el nodo anterior con el siguiente del elemento eliminado.Con esta instrucción liberamos la memoria del nodo eliminado y conservamos la conexión entre los elementos restantes.

21. Método para buscar un elemento (buscar(int valor))

Su función es localizar un número dentro de nuestra lista y mostrar la posición en la que se encuentra, recibimos el número que queremos buscar mediante el parámetro valor.

22. Recorrido de la lista (while (actual != NULL))

Utilizamos un ciclo while que se ejecuta mientras actual no sea igual a NULL, dentro del ciclo verificamos si el dato del nodo actual coincide con el valor que estamos buscando.

23. Avance al siguiente nodo (actual y posicion)

Si el valor del nodo actual no coincide con el que buscamos, avanzamos al siguiente nodo, con la primera instrucción continuamos el recorrido y con la segunda

aumentamos la posición en una unidad. Este procedimiento se repite hasta encontrar el elemento o llegar al final de la lista.

24. Método para mostrar todos los elementos (mostrar())

Su función es visualizar todos los números almacenados en nuestra lista, primero verificamos si inicio == NULL, sii se cumple esta condición, mostramos que la lista está vacía y utilizamos return para terminar el método.

25. Recorrido para mostrar los elementos (Nodo* actual = inicio)

Si existen elementos, declaramos un apuntador auxiliar llamado actual y lo inicializamos con inicio. Este apuntador nos permite recorrer todos los nodos sin modificar la estructura original.

26. Ciclo para mostrar los datos (while (actual != NULL))

Utilizamos un ciclo while que se ejecuta mientras existan nodos por recorrer.

Dentro del ciclo realizamos dos operaciones:

- Mostramos el dato del nodo actual mediante actual->dato.
- Avanzamos al siguiente nodo utilizando actual = actual->siguiente.

Además, utilizamos el símbolo -> para representar visualmente la conexión entre los elementos.

25. Representación del final de la lista (NULL)

Cuando terminamos de recorrer los nodos, mostramos la palabra NULL, esto indica que llegamos al final de nuestra lista y que el último nodo no apunta a ningún otro elemento.

26. Método para verificar si la lista está vacía (estaVacia())

Implementamos el método estaVacia(), que devuelve un valor de tipo bool. Si inicio es igual a NULL, el método devuelve true, indicando que la lista está vacía y si inicio contiene la dirección de un nodo, devuelve false, indicando que existen elementos almacenados.

27. Función principal (int main())

Declaramos la función principal del programa, desde donde iniciamos la ejecución, dentro de ella creamos un objeto llamado lista perteneciente a la clase Lista.

También declaramos dos variables enteras:

- opcion: almacena la opción seleccionada por el usuario.
- valor: almacena el número que queremos insertar, eliminar o buscar.